

Mini_Projet Système: Crible d'Ératosthène

Travail en binôme : JABER elham et ELCHAMAA mohamad , L3 inforamtique -IFA2

enseignant : Stéphane Rubini

UBO-Université de Bretagne Occidentale année 2025-2026

Phase 1:

Pendant la phase 1 ; il est demandé d'implémenter l'algorithme d'une façon séquentielle , la version séquentielle c'est une version qui marche étape par étape , donc la deuxième étape attend toujours la fin de la première pour commencer .

Pour $i = 2$ jusqu'à $\sqrt{N}$

si i est premier

marquer tous ses multiples

Pendant cette phase , on a implémenter l'algorithme en langage C et on a mesurer le temps en faisant augmenter N à chaque étape et en utilisant un shell run.sh .

donc , la phase1 est constitué d'un fichier .c qui porte le code c qui implémente l'algorithme , la fonction qui écrit dans le fichier , et la fonction qui calcule les temps , ensuite un script shell qui exécute les N nombre demandé partant de 10 à 10000000000,

Phase 2:

Pour la phase 2 : la méthode de parallélisation des calculs dans des threads est demandée , et donc c'est une façon de faire plusieurs taches en même temps .

```
#!/bin/bash

echo "N,CPU_TIME,WALL_TIME" > resultats.csv

./programme.exe 10 >> resultats.csv
./programme.exe 100 >> resultats.csv
./programme.exe 1000 >> resultats.csv
./programme.exe 10000 >> resultats.csv
./programme.exe 100000 >> resultats.csv
./programme.exe 1000000 >> resultats.csv
./programme.exe 10000000 >> resultats.csv
./programme.exe 100000000 >> resultats.csv
```

	A	B	C
1	N	CPU_TIME	WALL_TIME
2	10	0.000000	0.000000
3	100	0.000000	0.000001
4	1000	0.000000	0.000002
5	10000	0.000000	0.000017
6	100000	0.000000	0.000258
7	1000000	0.000000	0.004605

Wall time = temps réel écoulé

CPU time = temps de calcul réel du processeur

Comment ?

et donc imaginons on a un restaurant qui a un seul cuisinier qui fait les plats un par un , donc ce restaurant on peut dire qu'il possède un seul THREADS ;

ce restaurant choisit d'aller plus vite et donc recrute plusieurs cuisinier qui font plusieurs plats en même temps , ce qu'on peut imaginer en informatique : plusieurs Threads en même temps .

“un thread est une unité d'exécution qui permet à un programme de faire plusieurs choses en même temps”.

Pourquoi ?

1. Aller plus vite

2. Utiliser plusieurs cœurs du processeur
3. Diviser un gros travail

Dans cette phase , on a fait un fichier .c qui met le code c de threads et un deuxième fichier .sh qui exécute le programme avec le paramètre T égal à $2i$ avec $0 \leq i \leq 10$. Le paramètre N sera fixé à 10^9 .

Les résultats sont donnés dans un fichier résultat.csv :

	A	B	C	D
1	N	T	CPU_TIME	WALL_TIME
2	1000000000	1	19.219000	19.210326
3	1000000000	2	13.797000	13.791131
4	1000000000	4	12.780000	12.787560
5	1000000000	8	12.586000	12.581983
6	1000000000	16	13.381000	13.367179
7	1000000000	32	12.868000	12.866936
8	1000000000	64	11.933000	11.934760
9	1000000000	128	11.602000	11.602227
10	1000000000	256	11.807000	11.807335
11	1000000000	512	11.529000	11.529323
12	1000000000	1024	11.556000	11.556704

On observe une diminution significative du temps d'exécution lorsque le nombre de threads augmente de 1 à 8. Au-delà, le gain devient marginal et le temps tend à se stabiliser. Cela s'explique par le nombre limité de cœurs disponibles ainsi que par le coût de gestion des threads. Le parallélisme améliore donc les performances jusqu'à un certain seuil, après quoi les bénéfices deviennent négligeables.

Temps d'exécution en fonction du nombre de threads

